

Crypto AML Screening Tool

Author: Alroy Fernandes

Description: A Python-based AML screening tool that flags high-risk crypto wallet addresses using sanctions watchlist data, applies rule-based transaction monitoring filters, and generates risk-scored compliance reports — replicating a simplified crypto compliance workflow.

What this tool does:

1. Loads a synthetic OFAC-style sanctions watchlist of flagged wallet addresses
2. Simulates a batch of crypto transactions (BTC/ETH) across account-based and UTXO models
3. Screens each transaction against the watchlist and applies rule-based AML flags
4. Scores each transaction by risk level (Low / Medium / High / Critical)
5. Generates a flagged transaction report with audit trail output

Cell 1 — Import Libraries

```
In [1]: import pandas as pd
import numpy as np
import hashlib
import random
import datetime
import warnings
warnings.filterwarnings('ignore')

print("Libraries loaded successfully.")
```

Libraries loaded successfully.

Cell 2 — Build Sanctions Watchlist

Simulates an OFAC SDN-style watchlist of flagged crypto wallet addresses, with entity type and designation reason. In production, this would be loaded from OFAC's SDN CSV, Chainalysis feeds, or internal risk databases.

```
In [2]: # --- Sanctions Watchlist (OFAC SDN-style) ---
# Format mirrors real OFAC crypto address designations

sanctions_data = [
    # BTC addresses (P2PKH format)
    {"address": "1AeBzfpK3fJBMNxALKSBbpuFSSbmjMBFXa", "blockchain": "BTC", "entity": "Lazarus Group", "reason": "Sanctioned"},
    {"address": "1HB5XMLmzFVj8ALj6mfBsbfRoD4miY36v", "blockchain": "BTC", "entity": "Hydra Marketplace", "reason": "Sanctioned"},
    {"address": "149w62rY42aZBox8fGcmqNsXUzSStKeq8C", "blockchain": "BTC", "entity": "Garantex Exchange", "reason": "Sanctioned"},
    {"address": "1L2JsXHPMyuAa9ugvHGLwkdstCPUdemNCf", "blockchain": "BTC", "entity": "BitcoinFog Mixer", "reason": "Sanctioned"},
    {"address": "1KfZGvwZxsvSmemoCmEV75uqcNzYBjkhZ", "blockchain": "BTC", "entity": "Unknown - DPRK Nexus", "reason": "Sanctioned"},
    # ETH addresses
    {"address": "0xd882cFc20F52f2599D84b8e8D58C7FB62cfE344b", "blockchain": "ETH", "entity": "Tornado Cash", "reason": "Sanctioned"},
    {"address": "0x8576aCC5C05D6Ce88f4e49bf658dF0C62F91353C", "blockchain": "ETH", "entity": "FTX Hacker", "reason": "Sanctioned"},
    {"address": "0xA7e5d5A720f06526557c513402f2e6B5fA20b008", "blockchain": "ETH", "entity": "Ren Protocol Hack", "reason": "Sanctioned"},
    {"address": "0x098B716B8Aaf21512996dC57EB0615e2383E2f96", "blockchain": "ETH", "entity": "Lazarus Group ETH", "reason": "Sanctioned"},
    # Internal watchlist (higher risk, not publicly sanctioned yet)
    {"address": "1FfmbHfnpaZjKFvyilokTjJJusN455paPH", "blockchain": "BTC", "entity": "Unnamed Cluster A", "reason": "Internal"},
    {"address": "0x3f5CE5FBFe3E9af3971dD833D26bA9b5C936f0bE", "blockchain": "ETH", "entity": "Binance Hot Wallet", "reason": "Internal"}
]

watchlist_df = pd.DataFrame(sanctions_data)
sanctioned_addresses = set(watchlist_df["address"].str.lower())

print(f"Watchlist loaded: {len(watchlist_df)} flagged addresses")
print(f"OFAC SDN entries: {len(watchlist_df[watchlist_df['list']=='OFAC SDN'])}")
print(f"Internal Watchlist entries: {len(watchlist_df[watchlist_df['list']=='Internal Watchlist'])}")
print(f"Enhanced Monitoring: {len(watchlist_df[watchlist_df['list']=='Enhanced Monitoring'])}")
print()
watchlist_df
```

```
Watchlist loaded: 11 flagged addresses
OFAC SDN entries: 9
Internal Watchlist entries: 1
Enhanced Monitoring: 1
```

Out[2]:

	address	blockchain	entity	reason	list
0	1AeBzfpK3fjBMNxALKSBbpuFSSbmjMBFXa	BTC	Lazarus Group	DPRK state-sponsored cybercrime	OFAC SDN
1	1HB5XMLmzFVj8ALj6mfBsbiFroD4miY36v	BTC	Hydra Marketplace	Darknet drug and fraud marketplace	OFAC SDN
2	149w62rY42aZBox8fGcmqNsXUZSSStKeq8C	BTC	Garantex Exchange	Sanctions evasion — Russian entity	OFAC SDN
3	1L2JsXHPMYuAa9ugvHGLwkdstCPUdEmNCf	BTC	BitcoinFog Mixer	Crypto mixer / money laundering	OFAC SDN
4	1KfZGvwZxsvSmemoCmEV75uqcNzYBHjKHZ	BTC	Unknown — DPRK Nexus	Linked to Lazarus Group cluster	OFAC SDN
5	0xd882cFc20F52f2599D84b8e8D58C7FB62cfE344b	ETH	Tornado Cash	Crypto mixer / sanctions evasion	OFAC SDN
6	0x8576aCC5C05D6Ce88f4e49bf65BdF0C62F91353C	ETH	FTX Hacker	Suspected exchange exploit	OFAC SDN
7	0xA7e5d5A720f06526557c513402f2e6B5fA20b008	ETH	Ren Protocol Hack	DeFi exploit — funds laundering	OFAC SDN
8	0x098B716B8Aaf21512996dC57EB0615e2383E2f96	ETH	Lazarus Group ETH	DPRK — Ronin Bridge exploit	OFAC SDN
9	1FfmbHfnpaZjKFvyi1okTjjJusN455paPH	BTC	Unnamed Cluster A	High-velocity mixing pattern	Internal Watchlist
10	0x3f5CE5FBFe3E9af3971dD833D26bA9b5C936f0bE	ETH	Binance Hot Wallet	Monitoring — high volume exchange	Enhanced Monitoring

Cell 3 — Simulate Transaction Dataset

Generates a realistic batch of crypto transactions across BTC (UTXO model) and ETH (account-based model). A proportion of transactions are deliberately seeded with sanctioned addresses to simulate real AML hits.

In [3]:

```
random.seed(42)
np.random.seed(42)

def generate_btc_address():
    """Generate a plausible-looking BTC P2PKH address."""
    chars = "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"
    return "1" + "".join(random.choices(chars, k=33))

def generate_eth_address():
    """Generate a plausible-looking ETH address."""
    return "0x" + "".join(random.choices("0123456789abcdef", k=40))

def generate_tx_hash(seed):
    """Generate a deterministic transaction hash."""
    return hashlib.sha256(str(seed).encode()).hexdigest()

# Known sanctioned addresses to seed into transactions
sanctioned_btc = [a for a in watchlist_df["address"] if a.startswith("1") or a.startswith("3")]
sanctioned_eth = [a for a in watchlist_df["address"] if a.startswith("0x")]

# --- Transaction generation parameters ---
NUM_TRANSACTIONS = 200
base_date = datetime.datetime(2025, 1, 1)

# A pool of "regular" wallet addresses that gets reused across transactions.
# Real wallets transact repeatedly – reusing a fixed pool (rather than generating
# a brand-new random address every time) is what allows velocity-based rules
# (R4 layering / R5 aggregation) to actually trigger on repeat senders/receivers.
WALLET_POOL_SIZE = 40
btc_wallet_pool = [generate_btc_address() for _ in range(WALLET_POOL_SIZE)]
eth_wallet_pool = [generate_eth_address() for _ in range(WALLET_POOL_SIZE)]

# Designated "high-velocity" wallets – a small subset of the pool that will be
# deliberately used in rapid bursts to simulate layering/aggregation typologies.
layering_sender_btc = btc_wallet_pool[0]
layering_sender_eth = eth_wallet_pool[0]
aggregation_receiver_btc = btc_wallet_pool[1]
aggregation_receiver_eth = eth_wallet_pool[1]

transactions = []

# --- Inject deliberate layering burst: one BTC sender, many receivers, same day ---
```

```

burst_date = base_date + datetime.timedelta(days=40)
for j in range(4):
    transactions.append({
        "tx_id": generate_tx_hash(f"layer_btc_{j}")[16].upper(),
        "timestamp": burst_date + datetime.timedelta(hours=j*2),
        "blockchain": "BTC", "model": "UTXO", "asset": "BTC",
        "sender": layering_sender_btc,
        "receiver": random.choice(btc_wallet_pool),
        "amount_usd": round(np.random.uniform(2000, 8000), 2),
    })

# --- Inject deliberate aggregation burst: many BTC senders, one receiver, same day ---
agg_date = base_date + datetime.timedelta(days=75)
for j in range(4):
    transactions.append({
        "tx_id": generate_tx_hash(f"agg_btc_{j}")[16].upper(),
        "timestamp": agg_date + datetime.timedelta(hours=j*3),
        "blockchain": "BTC", "model": "UTXO", "asset": "BTC",
        "sender": random.choice(btc_wallet_pool),
        "receiver": aggregation_receiver_btc,
        "amount_usd": round(np.random.uniform(2000, 8000), 2),
    })

# --- Inject deliberate layering burst on ETH ---
burst_date_eth = base_date + datetime.timedelta(days=110)
for j in range(4):
    transactions.append({
        "tx_id": generate_tx_hash(f"layer_eth_{j}")[16].upper(),
        "timestamp": burst_date_eth + datetime.timedelta(hours=j*2),
        "blockchain": "ETH", "model": "Account-based", "asset": "ETH",
        "sender": layering_sender_eth,
        "receiver": random.choice(eth_wallet_pool),
        "amount_usd": round(np.random.uniform(1500, 6000), 2),
    })

# --- Inject deliberate aggregation burst on ETH ---
agg_date_eth = base_date + datetime.timedelta(days=135)
for j in range(4):
    transactions.append({
        "tx_id": generate_tx_hash(f"agg_eth_{j}")[16].upper(),
        "timestamp": agg_date_eth + datetime.timedelta(hours=j*3),
        "blockchain": "ETH", "model": "Account-based", "asset": "ETH",
        "sender": random.choice(eth_wallet_pool),
        "receiver": aggregation_receiver_eth,
        "amount_usd": round(np.random.uniform(1500, 6000), 2),
    })

# --- Generate the remaining transactions (random mix, drawing from the wallet pool) ---
NUM_REMAINING = NUM_TRANSACTIONS - len(transactions)

for i in range(NUM_REMAINING):
    blockchain = random.choice(["BTC", "BTC", "ETH"]) # BTC-heavy
    timestamp = base_date + datetime.timedelta(
        days=random.randint(0, 165),
        hours=random.randint(0, 23),
        minutes=random.randint(0, 59)
    )

    if blockchain == "BTC":
        # UTXO model: one sender, one receiver
        # Seed ~12% of transactions with sanctioned BTC addresses
        if random.random() < 0.12 and sanctioned_btc:
            role = random.choice(["sender", "receiver"])
            flagged_addr = random.choice(sanctioned_btc)
            sender = flagged_addr if role == "sender" else random.choice(btc_wallet_pool)
            receiver = flagged_addr if role == "receiver" else random.choice(btc_wallet_pool)
        else:
            sender = random.choice(btc_wallet_pool)
            receiver = random.choice(btc_wallet_pool)

        amount_usd = round(np.random.lognormal(mean=8.5, sigma=2.2), 2) # skewed: occasional large txns
        asset = "BTC"
        model = "UTXO"

    else:
        # Account-based model (ETH)
        if random.random() < 0.10 and sanctioned_eth:
            role = random.choice(["sender", "receiver"])
            flagged_addr = random.choice(sanctioned_eth)
            sender = flagged_addr if role == "sender" else random.choice(eth_wallet_pool)
            receiver = flagged_addr if role == "receiver" else random.choice(eth_wallet_pool)
        else:
            sender = random.choice(eth_wallet_pool)

```

```
receiver = random.choice(eth_wallet_pool)

amount_usd = round(np.random.lognormal(mean=7.5, sigma=2.5), 2)
asset = random.choice(["ETH", "ETH", "USDT", "USDC"])
model = "Account-based"

transactions.append({
    "tx_id": generate_tx_hash(f"{i}_{timestamp}")[:16].upper(),
    "timestamp": timestamp,
    "blockchain": blockchain,
    "model": model,
    "asset": asset,
    "sender": sender,
    "receiver": receiver,
    "amount_usd": amount_usd,
})

txn_df = pd.DataFrame(transactions).sort_values("timestamp").reset_index(drop=True)

print(f"Transaction dataset generated: {len(txn_df)} transactions")
print(f"  BTC (UTX0):          {len(txn_df[txn_df['blockchain']=='BTC'])}")
print(f"  ETH (Account-based):    {len(txn_df[txn_df['blockchain']=='ETH'])}")
print(f"  Total volume (USD):      ${txn_df['amount_usd'].sum():.0f}")
print(f"  Date range:             {txn_df['timestamp'].min().date()} to {txn_df['timestamp'].max().date()}")
print()
txn_df.head(10)
```

Transaction dataset generated: 200 transactions
BTC (UTX0): 128
ETH (Account-based): 72
Total volume (USD): \$6,353,402
Date range: 2025-01-01 to 2025-06-15

Out[3]:

	tx_id	timestamp	blockchain	model	asset	sender	
0	70F8F5F94EAC57ED	2025-01-01 22:36:00	ETH	Account-based	ETH	0x8370fb1ee8b5f112d196f345cbb5413c922768f3	0x406
1	DD0BE3D4C38318B1	2025-01-02 01:34:00	BTC	UTXO	BTC	1MkdmCYvSh8ycEAYZ6zuT7qViWGqyFYPvW	
2	CB622ADF951D3769	2025-01-02 16:47:00	BTC	UTXO	BTC	1R6VQx2NSxr6gYyMQC8rTfeb2nF8Z4mDDs	
3	47157B0CDFB1C773	2025-01-03 03:42:00	ETH	Account-based	ETH	0xc78707cb6db9051fe1e06ccfa6f6de6aa8a5288b	0x2
4	55BF7C1879FB38A8	2025-01-03 15:58:00	ETH	Account-based	ETH	0xab93383f443b55ec41a6fdcf44b574738eb2993a	0x2ed5
5	2927B09342EC3160	2025-01-03 20:59:00	BTC	UTXO	BTC	1R6VQx2NSxr6gYyMQC8rTfeb2nF8Z4mDDs	
6	F6077892EFBC7EEE	2025-01-04 16:06:00	BTC	UTXO	BTC	1AeBzfpK3fjBMNxALKSBbpuFSSbmjMBFXa	
7	4F1EF99D5F7CAAFB	2025-01-05 23:46:00	ETH	Account-based	ETH	0x9d3cb87d5f0ef1f7393e8c6854871f7de569324e	0x31b7
8	93B37C85E64D5FDD	2025-01-06 04:37:00	BTC	UTXO	BTC	115tuYqa98JuoruDF6ntQc9vsypt2jLvoss	
9	74ABDCD8C42F89F3	2025-01-06 10:52:00	BTC	UTXO	BTC	1DVjyXH6CEB1XGyZh8sVsaUSB3wUpQ5d49	

Cell 4 — AML Rule Engine

Applies five rule-based AML filters, mirroring standard crypto compliance logic. Each rule contributes a weighted score; transactions are then bucketed into Low / Medium / High / Critical based on cumulative score.

Rule	Trigger	Rationale	Score Weight
R1	Sender or receiver on sanctions watchlist	Direct OFAC/SDN hit	35
R2	Transaction ≥ \$10,000 USD	CTR threshold (FinCEN)	20
R3	Transaction ≥ \$3,000 but < \$10,000	Structuring / smurfing risk	12
R4	Same sender → multiple receivers within 24h (≥3 txns)	Layering pattern	18
R5	Same receiver from multiple senders within 24h (≥3 txns)	Aggregation / consolidation	18

Risk bands: Critical (≥45) | High (25-44) | Medium (10-24) | Low (<10, unflagged)

```

In [4]: def screen_transactions(txn_df, watchlist_df, sanctioned_addresses):
    """
    Apply AML rule-based screening to a transaction dataset.
    Returns the dataframe with risk flags and scores appended.
    """
    df = txn_df.copy()

    # --- Pre-compute: velocity analysis (24-hour window) ---
    df["timestamp"] = pd.to_datetime(df["timestamp"])
    df_sorted = df.sort_values("timestamp")

    # For each transaction, count how many txns the same sender made within ±24h
    sender_velocity = {}
    receiver_velocity = {}

    for idx, row in df_sorted.iterrows():
        window_start = row["timestamp"] - datetime.timedelta(hours=24)
        window_end = row["timestamp"] + datetime.timedelta(hours=24)
        same_sender_window = df_sorted[
            (df_sorted["sender"] == row["sender"]) &
            (df_sorted["timestamp"] >= window_start) &
            (df_sorted["timestamp"] <= window_end)
        ]
        same_receiver_window = df_sorted[
            (df_sorted["receiver"] == row["receiver"]) &
            (df_sorted["timestamp"] >= window_start) &
            (df_sorted["timestamp"] <= window_end)
        ]
        sender_velocity[idx] = len(same_sender_window)
        receiver_velocity[idx] = len(same_receiver_window)

    df["sender_24h_txn_count"] = df.index.map(sender_velocity)
    df["receiver_24h_txn_count"] = df.index.map(receiver_velocity)

    # --- Apply Rules ---
    flags = []
    reasons = []

    for _, row in df.iterrows():
        row_flags = []

        # R1 - Sanctions watchlist hit
        sender_hit = row["sender"].lower() in sanctioned_addresses
        receiver_hit = row["receiver"].lower() in sanctioned_addresses
        if sender_hit:
            entity = watchlist_df[watchlist_df["address"].str.lower() == row["sender"].lower()][0]["entity"].value
            row_flags.append(f"R1-SENDER: Sanctioned address ({entity})")
        if receiver_hit:
            entity = watchlist_df[watchlist_df["address"].str.lower() == row["receiver"].lower()][0]["entity"].value
            row_flags.append(f"R1-RECEIVER: Sanctioned address ({entity})")

        # R2 - Large transaction (CTR threshold)
        if row["amount_usd"] >= 10_000:
            row_flags.append(f"R2: Large transaction (${row['amount_usd']:,.0f} - CTR threshold)")

        # R3 - Structuring band
        elif 3_000 <= row["amount_usd"] < 10_000:
            row_flags.append(f"R3: Structuring band (${row['amount_usd']:,.0f})")

        # R4 - High sender velocity (layering)
        if row["sender_24h_txn_count"] >= 3:
            row_flags.append(f"R4: High sender velocity ({row['sender_24h_txn_count']} txns/24h - layering pattern)")

        # R5 - High receiver velocity (aggregation)
        if row["receiver_24h_txn_count"] >= 3:
            row_flags.append(f"R5: High receiver velocity ({row['receiver_24h_txn_count']} txns/24h - aggregation)")

        flags.append(len(row_flags) > 0)
        reasons.append(" | ".join(row_flags) if row_flags else "Clean")

    df["flagged"] = flags
    df["flag_reasons"] = reasons

    # --- Risk Scoring ---
    # Weights are spread so that combinations of rules land across all four
    # bands (Low / Medium / High / Critical) rather than jumping straight from
    # Medium to Critical. A standalone sanctions hit (R1) is High on its own;
    # it only escalates to Critical when paired with another red flag
    # (large value, structuring, or velocity), reflecting that a sanctions
    # match plus a second typology indicator is materially higher risk than
    # a sanctions match in isolation.
    def score_risk(row):

```

```

        if not row["flagged"]:
            return "Low", 0
        score = 0
        reasons = row["flag_reasons"]
        if "R1" in reasons: score += 35      # Sanctions hit – serious on its own
        if "R2" in reasons: score += 20      # Large transaction (CTR threshold)
        if "R3" in reasons: score += 12      # Structuring band
        if "R4" in reasons: score += 18      # Layering (velocity)
        if "R5" in reasons: score += 18      # Aggregation (velocity)
        if score >= 45: return "Critical", score
        elif score >= 25: return "High", score
        elif score >= 10: return "Medium", score
        else: return "Low", score

    df[["risk_level", "risk_score"]] = df.apply(
        lambda r: pd.Series(score_risk(r)), axis=1
    )

    return df

# --- Run the screening engine ---
screened_df = screen_transactions(txn_df, watchlist_df, sanctioned_addresses)

print("=" * 55)
print("          AML SCREENING – SUMMARY RESULTS")
print("=" * 55)
print(f"  Total transactions screened:  {len(screened_df)}")
print(f"  Flagged transactions:           {screened_df['flagged'].sum()} ({screened_df['flagged'].mean()*100:.1f}%)"
print(f"  Clean transactions:             {(~screened_df['flagged']).sum()}")
print()
print("  Risk Level Breakdown:")
for level in ["Critical", "High", "Medium", "Low"]:
    count = len(screened_df[screened_df["risk_level"] == level])
    print(f"    {level:<10} {count:>4} transactions")
print("=" * 55)

```

```

=====
                        AML SCREENING – SUMMARY RESULTS
=====
Total transactions screened:   200
Flagged transactions:          122 (61.0%)
Clean transactions:           78

Risk Level Breakdown:
Critical      11 transactions
High          18 transactions
Medium        93 transactions
Low           78 transactions
=====

```

Cell 5 — Flagged Transactions Report

Displays flagged transactions by risk level, with full audit trail detail.

```

In [5]: # Display flagged transactions sorted by risk score
flagged_df = screened_df[screened_df["flagged"]].sort_values(
    ["risk_score", "amount_usd"], ascending=[False, False]
).reset_index(drop=True)

report_cols = [
    "tx_id", "timestamp", "blockchain", "asset",
    "amount_usd", "risk_level", "risk_score", "flag_reasons"
]

pd.set_option("display.max_colwidth", 80)
pd.set_option("display.max_rows", 60)

print(f"Flagged Transactions – {len(flagged_df)} alerts generated")
print()
flagged_df[report_cols]

```

Flagged Transactions – 122 alerts generated

Out[5]:

	tx_id	timestamp	blockchain	asset	amount_usd	risk_level	risk_score	flag_reasons
0	D098EFDF812F1AB6	2025-02-18 22:21:00	BTC	BTC	107596.94	Critical	55	R1-RECEIVER: Sanctioned address (Lazarus Group) R2: Large transaction (\$10...
1	447D4B4FD2F27637	2025-05-13 07:57:00	BTC	BTC	44701.95	Critical	55	R1-RECEIVER: Sanctioned address (Unnamed Cluster A) R2: Large transaction ...
2	CB622ADF951D3769	2025-01-02 16:47:00	BTC	BTC	17873.70	Critical	55	R1-RECEIVER: Sanctioned address (Unknown — DPRK Nexus) R2: Large transacti...
3	055D7E141BAF0FB6	2025-04-29 20:35:00	ETH	ETH	14117.22	Critical	55	R1-SENDER: Sanctioned address (Ren Protocol Hack) R2: Large transaction (\$...
4	36586AE8537F5001	2025-04-12 18:06:00	ETH	USDT	13784.70	Critical	55	R1-RECEIVER: Sanctioned address (Binance Hot Wallet) R2: Large transaction...
...	...	...	...	...	...	...	...	...
117	6B9D1BFF84CE62DC	2025-05-03 13:41:00	BTC	BTC	3446.70	Medium	12	R3: Structuring band (\$3,447)
118	021E2DEAD86199B9	2025-05-27 14:05:00	BTC	BTC	3218.94	Medium	12	R3: Structuring band (\$3,219)
119	7D24FC575661E5A6	2025-03-30 17:26:00	ETH	ETH	3047.75	Medium	12	R3: Structuring band (\$3,048)
120	3ADBBEBBBDDBF7B7	2025-02-23 05:36:00	BTC	BTC	3031.22	Medium	12	R3: Structuring band (\$3,031)
121	2DE5B7800C9EEF75	2025-04-07 11:37:00	BTC	BTC	3006.05	Medium	12	R3: Structuring band (\$3,006)

122 rows × 8 columns

Cell 6 — Sanctions Hit Detail

Deep-dive on transactions that directly hit the OFAC/SDN watchlist — highest priority for SAR filing.

In [6]:

```
sanctions_hits = screened_df[
    screened_df["flag_reasons"].str.contains("R1", na=False)
].sort_values("risk_score", ascending=False).reset_index(drop=True)

print("=" * 60)
print("  SANCTIONS WATCHLIST HITS — PRIORITY ALERTS")
print("=" * 60)
print(f"  {len(sanctions_hits)} transaction(s) matched a sanctioned address")
print()

for _, row in sanctions_hits.iterrows():
    # Identify which side was sanctioned and enrich with watchlist detail
    sender_match = row["sender"].lower() in sanctioned_addresses
    receiver_match = row["receiver"].lower() in sanctioned_addresses

    def get_watchlist_info(addr):
        match = watchlist_df[watchlist_df["address"].str.lower() == addr.lower()]
        if not match.empty:
            m = match.iloc[0]
            return f"{m['entity']} | {m['reason']} | List: {m['list']}"
        return "Not found"

    print(f"  TX ID:      {row['tx_id']}")
    print(f"  Timestamp:  {row['timestamp']}")
    print(f"  Chain:      {row['blockchain']} ({row['model']})")
    print(f"  Asset:      {row['asset']}")
    print(f"  Amount:     ${row['amount_usd']:.2f} USD")
```

```

print(f" Risk Level: {row['risk_level']} (score: {row['risk_score']})")
print(f" Sender: {row['sender']}")
if sender_match:
    print(f" ⚠ SANCTIONED SENDER – {get_watchlist_info(row['sender'])}")
print(f" Receiver: {row['receiver']}")
if receiver_match:
    print(f" ⚠ SANCTIONED RECEIVER – {get_watchlist_info(row['receiver'])}")
print(f" Flags: {row['flag_reasons']}")
print("-" * 60)

```

===== SANCTIONS WATCHLIST HITS – PRIORITY ALERTS =====

20 transaction(s) matched a sanctioned address

TX ID: CB622ADF951D3769
 Timestamp: 2025-01-02 16:47:00
 Chain: BTC (UTX0)
 Asset: BTC
 Amount: \$17,873.70 USD
 Risk Level: Critical (score: 55)
 Sender: 1R6VQx2NSxr6gYyMQC8rTfeb2nF8Z4mDDs
 Receiver: 1KfZGvwZxsvSmemoCmEV75uqcNzYBHjkHZ
 ⚠ SANCTIONED RECEIVER – Unknown – DPRK Nexus | Linked to Lazarus Group cluster | List: OFAC SDN
 Flags: R1-RECEIVER: Sanctioned address (Unknown – DPRK Nexus) | R2: Large transaction (\$17,874 – CTR threshold)

 TX ID: 36586AE8537F5001
 Timestamp: 2025-04-12 18:06:00
 Chain: ETH (Account-based)
 Asset: USDT
 Amount: \$13,784.70 USD
 Risk Level: Critical (score: 55)
 Sender: 0x2ed50729d60a2324e0934680423e8accfc587e86
 Receiver: 0x3f5CE5FBFe3E9af3971dD833D26bA9b5C936f0bE
 ⚠ SANCTIONED RECEIVER – Binance Hot Wallet | Monitoring – high volume exchange | List: Enhanced Monitoring
 Flags: R1-RECEIVER: Sanctioned address (Binance Hot Wallet) | R2: Large transaction (\$13,785 – CTR threshold)

 TX ID: 379179059AA5BEAC
 Timestamp: 2025-04-06 22:06:00
 Chain: ETH (Account-based)
 Asset: USDC
 Amount: \$13,063.53 USD
 Risk Level: Critical (score: 55)
 Sender: 0x098B716B8Aaf21512996dC57EB0615e2383E2f96
 ⚠ SANCTIONED SENDER – Lazarus Group ETH | DPRK – Ronin Bridge exploit | List: OFAC SDN
 Receiver: 0x4314693d2595c78740f8f3507ea78d6ebc569381
 Flags: R1-SENDER: Sanctioned address (Lazarus Group ETH) | R2: Large transaction (\$13,064 – CTR threshold)

 TX ID: D098EFDf812F1AB6
 Timestamp: 2025-02-18 22:21:00
 Chain: BTC (UTX0)
 Asset: BTC
 Amount: \$107,596.94 USD
 Risk Level: Critical (score: 55)
 Sender: 1mfBmVkmTvZddsd94SJG4WKT4q5srcWTzn
 Receiver: 1AeBzfpk3fJBMNxALKSBbpuFSSbmjMBFXa
 ⚠ SANCTIONED RECEIVER – Lazarus Group | DPRK state-sponsored cybercrime | List: OFAC SDN
 Flags: R1-RECEIVER: Sanctioned address (Lazarus Group) | R2: Large transaction (\$107,597 – CTR threshold)

 TX ID: 447D4B4FD2F27637
 Timestamp: 2025-05-13 07:57:00
 Chain: BTC (UTX0)
 Asset: BTC
 Amount: \$44,701.95 USD
 Risk Level: Critical (score: 55)
 Sender: 1ZJz7mcnEXTSrZJdcjwDDfAB51TbHEihTg
 Receiver: 1FfmbHfnpaZjKFvyilokTjJJusN455paPH
 ⚠ SANCTIONED RECEIVER – Unnamed Cluster A | High-velocity mixing pattern | List: Internal Watchlist
 Flags: R1-RECEIVER: Sanctioned address (Unnamed Cluster A) | R2: Large transaction (\$44,702 – CTR threshold)

 TX ID: 055D7E141BAF0FB6
 Timestamp: 2025-04-29 20:35:00
 Chain: ETH (Account-based)
 Asset: ETH
 Amount: \$14,117.22 USD
 Risk Level: Critical (score: 55)
 Sender: 0xA7e5d5A720f06526557c513402f2e6B5fA20b008
 ⚠ SANCTIONED SENDER – Ren Protocol Hack | DeFi exploit – funds laundering | List: OFAC SDN
 Receiver: 0x8914fb8cd1fa7a5eca2f6e01225b5fed4a824882
 Flags: R1-SENDER: Sanctioned address (Ren Protocol Hack) | R2: Large transaction (\$14,117 – CTR threshold)

)

TX ID: 14263BE332793884
Timestamp: 2025-04-15 09:10:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$9,428.24 USD
Risk Level: Critical (score: 47)
Sender: 1R6VQx2NSxr6gYyMQC8rTfeb2nF8Z4mDDs
Receiver: 1HB5XMLmzFVj8ALj6mfBsbifRoD4miY36v
△ SANCTIONED RECEIVER – Hydra Marketplace | Darknet drug and fraud marketplace | List: OFAC SDN
Flags: R1-RECEIVER: Sanctioned address (Hydra Marketplace) | R3: Structuring band (\$9,428)

TX ID: 092B0C1145783794
Timestamp: 2025-05-08 23:41:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$9,541.48 USD
Risk Level: Critical (score: 47)
Sender: 1E2KGDwsKfPvTGFZGauQDzW637dnR4PzXy
Receiver: 1L2JsXHPMyAa9ugvHGLwkdstCPUDemNCf
△ SANCTIONED RECEIVER – BitcoinFog Mixer | Crypto mixer / money laundering | List: OFAC SDN
Flags: R1-RECEIVER: Sanctioned address (BitcoinFog Mixer) | R3: Structuring band (\$9,541)

TX ID: 4030A8B39D901EE5
Timestamp: 2025-05-27 17:22:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$7,377.53 USD
Risk Level: Critical (score: 47)
Sender: 1GKY9Ehh4QYRCRuahrp1MkrxRkYbDDS2L
Receiver: 1KfZGvwZxsvSmemoCmEV75uqcNzYBHjkhZ
△ SANCTIONED RECEIVER – Unknown – DPRK Nexus | Linked to Lazarus Group cluster | List: OFAC SDN
Flags: R1-RECEIVER: Sanctioned address (Unknown – DPRK Nexus) | R3: Structuring band (\$7,378)

TX ID: DFAE20F8F8437CCF
Timestamp: 2025-05-19 23:18:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$9,446.19 USD
Risk Level: Critical (score: 47)
Sender: 1UQgVu55c4GdYKzXTc6hremiDTELTR6RfN
Receiver: 1KfZGvwZxsvSmemoCmEV75uqcNzYBHjkhZ
△ SANCTIONED RECEIVER – Unknown – DPRK Nexus | Linked to Lazarus Group cluster | List: OFAC SDN
Flags: R1-RECEIVER: Sanctioned address (Unknown – DPRK Nexus) | R3: Structuring band (\$9,446)

TX ID: 8D42DDB4B08B2353
Timestamp: 2025-03-08 20:20:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$8,661.22 USD
Risk Level: Critical (score: 47)
Sender: 1FfmbHfnpaZjKFvyilokTjJJusN455paPH
△ SANCTIONED SENDER – Unnamed Cluster A | High-velocity mixing pattern | List: Internal Watchlist
Receiver: 1vndfwRYeup5AJkaH8ghwWV53SKF6xqaxz
Flags: R1-SENDER: Sanctioned address (Unnamed Cluster A) | R3: Structuring band (\$8,661)

TX ID: F6077892EFBC7EEE
Timestamp: 2025-01-04 16:06:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$474.82 USD
Risk Level: High (score: 35)
Sender: 1AeBzfpK3fJBMNxALKSBbpuFSSbmjMBFXa
△ SANCTIONED SENDER – Lazarus Group | DPRK state-sponsored cybercrime | List: OFAC SDN
Receiver: 1r1ligYGe7STxsGWBusJec9mYnX1K2vsqJ4
Flags: R1-SENDER: Sanctioned address (Lazarus Group)

TX ID: 95E66FB070F388D2
Timestamp: 2025-02-15 08:35:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$1,020.89 USD
Risk Level: High (score: 35)
Sender: 149w62rY42aZBox8fGcmqNsXUzSSStKeq8C
△ SANCTIONED SENDER – Garantex Exchange | Sanctions evasion – Russian entity | List: OFAC SDN
Receiver: 16dsShu3oJN9XZnA5scEu9TFFlougASMbe
Flags: R1-SENDER: Sanctioned address (Garantex Exchange)

TX ID: 656B2A4D6023D4D8
Timestamp: 2025-02-09 01:23:00
Chain: BTC (UTX0)
Asset: BTC

Amount: \$1,307.99 USD
Risk Level: High (score: 35)
Sender: 149w62rY42aZBox8fGcmqNsXUzSSStKeq8C
△ SANCTIONED SENDER – Garantex Exchange | Sanctions evasion – Russian entity | List: OFAC SDN
Receiver: 1tToeKU947uLiWBFSSXANHQLbne46gHifu
Flags: R1-SENDER: Sanctioned address (Garantex Exchange)

TX ID: A9C38632724AEDFF
Timestamp: 2025-03-14 09:15:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$2,587.05 USD
Risk Level: High (score: 35)
Sender: 1VSGk7RHgVf3Pb1JD8FL1kBPhWqo5r32vs
Receiver: 1KfZGvwZxsvSmemoCmEV75uqcNzYBHjkhZ
△ SANCTIONED RECEIVER – Unknown – DPRK Nexus | Linked to Lazarus Group cluster | List: OFAC SDN
Flags: R1-RECEIVER: Sanctioned address (Unknown – DPRK Nexus)

TX ID: C56CA911450BB051
Timestamp: 2025-03-31 17:24:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$213.91 USD
Risk Level: High (score: 35)
Sender: 1GKY9Ehh4QYRCRuahrmP1MkrxRkYbDD52L
Receiver: 1KfZGvwZxsvSmemoCmEV75uqcNzYBHjkhZ
△ SANCTIONED RECEIVER – Unknown – DPRK Nexus | Linked to Lazarus Group cluster | List: OFAC SDN
Flags: R1-RECEIVER: Sanctioned address (Unknown – DPRK Nexus)

TX ID: 9B87C2DEEA784ECB
Timestamp: 2025-04-17 19:41:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$2,106.58 USD
Risk Level: High (score: 35)
Sender: 16dsShu3oJN9XZnA5scEu9TFFlouGASMbe
Receiver: 1HB5XMLmzFVj8ALj6mfBsbifRoD4miY36v
△ SANCTIONED RECEIVER – Hydra Marketplace | Darknet drug and fraud marketplace | List: OFAC SDN
Flags: R1-RECEIVER: Sanctioned address (Hydra Marketplace)

TX ID: AC8FD1101552C300
Timestamp: 2025-04-25 05:43:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$327.70 USD
Risk Level: High (score: 35)
Sender: 1FfmbHfnpaZjKFvyilokTjJJusN455paPH
△ SANCTIONED SENDER – Unnamed Cluster A | High-velocity mixing pattern | List: Internal Watchlist
Receiver: 1UQgVu55c4GdYKzXTc6hremiDTELTR6RfN
Flags: R1-SENDER: Sanctioned address (Unnamed Cluster A)

TX ID: 08B281B3E64EFDc8
Timestamp: 2025-05-11 23:39:00
Chain: BTC (UTX0)
Asset: BTC
Amount: \$2,074.26 USD
Risk Level: High (score: 35)
Sender: 1AeBzfpK3fJBMNxALKSBbpuFSSbmjMBFXa
△ SANCTIONED SENDER – Lazarus Group | DPRK state-sponsored cybercrime | List: OFAC SDN
Receiver: 1L9u1r98FBf21nEKB4kXkUnW7Ww3nsXTx4
Flags: R1-SENDER: Sanctioned address (Lazarus Group)

TX ID: 823B665D2572CAC2
Timestamp: 2025-05-21 00:52:00
Chain: ETH (Account-based)
Asset: ETH
Amount: \$1,657.76 USD
Risk Level: High (score: 35)
Sender: 0x4158e2cff174d5803949797c3ec30635def0adb1
Receiver: 0x3f5CE5FBFe3E9af3971dD833D26bA9b5C936f0bE
△ SANCTIONED RECEIVER – Binance Hot Wallet | Monitoring – high volume exchange | List: Enhanced Monitoring
Flags: R1-RECEIVER: Sanctioned address (Binance Hot Wallet)

Cell 7 — Risk Dashboard

Summary statistics and volume analysis across the screened batch.

```
In [7]: print("=" * 60)
print("          COMPLIANCE RISK DASHBOARD")
print("=" * 60)
```

```
# Volume at risk by risk level
risk_summary = screened_df.groupby("risk_level").agg(
    transaction_count = ("tx_id", "count"),
    total_volume_usd = ("amount_usd", "sum"),
    avg_transaction = ("amount_usd", "mean"),
    max_transaction = ("amount_usd", "max"),
).round(2).reindex(["Critical", "High", "Medium", "Low"])

print("\n Volume at Risk by Risk Level:")
print(f"  {'Level':<12} {'Count':>6} {'Total Volume (USD)':>20} {'Avg Txn':>14} {'Max Txn':>14}")
print(" " + "-" * 68)
for level, row in risk_summary.iterrows():
    if pd.isna(row['transaction_count']):
        print(f"  {level:<12} {'0':>6} {'$0.00':>20} {'$0.00':>14} {'$0.00':>14}")
    else:
        print(f"  {level:<12} {int(row['transaction_count']):>6} "
              f"${row['total_volume_usd']:>19,.2f} "
              f"${row['avg_transaction']:>13,.2f} "
              f"${row['max_transaction']:>13,.2f}")

# Flag type breakdown
print("\n Alerts by Rule:")
rule_labels = {
    "R1": "R1 – Sanctions Watchlist Hit",
    "R2": "R2 – Large Transaction (≥$10k)",
    "R3": "R3 – Structuring Band ($3k–$10k)",
    "R4": "R4 – High Sender Velocity (layering)",
    "R5": "R5 – High Receiver Velocity (aggregation)",
}
for code, label in rule_labels.items():
    count = screened_df["flag_reasons"].str.contains(code, na=False).sum()
    print(f"  {label:<45} {count:>4} alert(s)")

# Blockchain breakdown of flagged txns
print("\n Flagged Transactions by Blockchain:")
chain_flags = screened_df[screened_df["flagged"]].groupby("blockchain")["tx_id"].count()
for chain, count in chain_flags.items():
    print(f"  {chain:<10} {count:>4} flagged transactions")

print("\n" + "=" * 60)
```

COMPLIANCE RISK DASHBOARD

Volume at Risk by Risk Level:				
Level	Count	Total Volume (USD)	Avg Txn	Max Txn
Critical	11	\$ 255,592.70	\$ 23,235.70	\$ 107,596.94
High	18	\$ 62,905.39	\$ 3,494.74	\$ 7,704.29
Medium	93	\$ 5,965,734.97	\$ 64,147.69	\$ 1,952,013.90
Low	78	\$ 69,168.90	\$ 886.78	\$ 2,990.79

Alerts by Rule:

R1 – Sanctions Watchlist Hit	20 alert(s)
R2 – Large Transaction (≥\$10k)	54 alert(s)
R3 – Structuring Band (\$3k–\$10k)	52 alert(s)
R4 – High Sender Velocity (layering)	8 alert(s)
R5 – High Receiver Velocity (aggregation)	8 alert(s)

Flagged Transactions by Blockchain:

BTC	82 flagged transactions
ETH	40 flagged transactions

Cell 8 — Export Compliance Report (CSV)

Exports the full screened dataset and a separate flagged-only report with audit timestamp — standard output for a compliance review or SAR referral workflow.

```
In [8]: run_timestamp = datetime.datetime.utcnow().strftime("%Y%m%d_%H%M%S")
```

```
# Full screened dataset
full_output_path = f"aml_screened_full_{run_timestamp}.csv"
flagged_output_path = f"aml_flagged_report_{run_timestamp}.csv"

screened_df.to_csv(full_output_path, index=False)

flagged_df_export = screened_df[
    screened_df["flagged"]
].sort_values(["risk_score", "amount_usd"], ascending=[False, False])
```

```
flagged_df_export.to_csv(flagged_output_path, index=False)
```

```
print(f"Reports exported:")
print(f"  Full screened dataset:   {full_output_path}  ({len(screened_df)} rows)")
print(f"  Flagged transactions:      {flagged_output_path}  ({len(flagged_df_export)} rows)")
print()
print(f"  Audit timestamp (UTC):       {run_timestamp}")
print(f"  Screening rules applied: R1 (Sanctions), R2 (CTR), R3 (Structuring), R4 (Layering), R5 (Aggregation)")
print(f"  Watchlist version:           OFAC SDN + Internal – loaded at runtime")
```

Reports exported:

Full screened dataset: aml_screened_full_20260619_171837.csv (200 rows)
Flagged transactions: aml_flagged_report_20260619_171837.csv (122 rows)

Audit timestamp (UTC): 20260619_171837
Screening rules applied: R1 (Sanctions), R2 (CTR), R3 (Structuring), R4 (Layering), R5 (Aggregation)
Watchlist version: OFAC SDN + Internal – loaded at runtime